Задание 5

Вариант 1

Напишите следующий метод, отыскивающий максимальное и минимальное значения в массиве. Метод возвращает другой массив, состоящий из найденных двух элементов, расположенных в порядке убывания.

```
public static Comparable[] maxAndMin(Comparable[] array)
```

Если в исходном массиве меньше двух элементов функция выбрасывает исключение. Функция должна работать с любыми данными, реализующими интерфейс Comparable.
Нужно протестировать функцию на данных системного типа (например, числа или строки) и данных собственного класса. В последнем случае класс должен реализовать интерфейс Comparable и иметь переопределённый метод compareTo().

Вариант 2

Пусть имеется следующий класс MyStack. Нужно реализовать в нем интерфейс Clonable и метод clone() так, чтобы он возвращал глубокую копию объекта (то есть самого стека). Протестировать работу стека, показать, что склонированный стек содержит собственный независимый ArrayList .

```
import java.util.ArrayList;

public class MyStack {

   private ArrayList< Object> list = new ArrayList<>();

   public boolean isEmpty() {

       return list.isEmpty();

   }

   public int getSize() {

       return list.size();

   }

   public Object peek() {

       return list.get(getSize() - 1);

   }
```

```
    public Object pop() {
        Object o = list.get(getSize() - 1);
        list.remove(getSize() - 1);
        return o;
    }
    public void push(Object o) {
        list.add(o);
    }
   @Override /** Переопределяет метод toString класса Object */
    public String toString() {
        return "стек: " + list.toString();
    }
}
```

УКАЗАНИЕ. Можно посмотреть дополнительные материалы с описанием стандартных интерфейсов Comparable и Clonable и базовых приёмов работы с ними – см. дополнительные материалы к заданию.

Вот так нужно определить новый класс стека, чтобы он реализовал интерфейс Clonable

```
class MyStack implements Cloneable {
```

Вот так определяется метод стека clone()

```
public Object clone() {
try {
      MyStack c = (MyStack) super.clone();
      c.list = (ArrayList<Object>) this.list.clone();
      return c;
    } catch (CloneNotSupportedException ex) {
      return null;
    }
  }
```